

Clasificación Automática de Calidad y Tamaño de Frutas

Mediante Visión por Computadora y Aprendizaje Automático — Metodología CRISP-DM

Angy Hurtado Osorio · Hideki Tamura Hernández · David Vergara Laverde

Repositorio del código: <https://github.com/LissaAN1/fruit-quality-classification/tree/main>

Resumen (Abstract)

Se presenta un sistema de clasificación multi-salida que predice simultáneamente la **calidad** (Mala / Regular / Buena) y el **tamaño** (Pequeño / Mediano / Grande) de seis tipos de frutas a partir de imágenes estáticas, usando aprendizaje automático y visión por computadora. El proyecto aplica la metodología CRISP-DM de principio a fin: desde la construcción del dataset hasta el despliegue en una aplicación web interactiva.

El dataset comprende 28 395 imágenes originales de seis frutas con desbalanceo notable (Good: 51.4%, Bad: 32.2%, Regular: 16.4%, $IR \approx 3.14$). Se entrenaron y compararon dos familias de modelos: (1) **ML tradicional** — Random Forest, SVM lineal y XGBoost —, con XGBoost liderando con F1-macro promedio de **0.9616**; y (2) una **CNN bifurcada desde cero** con accuracy de **78 % en calidad** y **93 % en tamaño**. Los modelos se despliegan en una aplicación Streamlit con predicción en tiempo real.

I. Introducción

En mercados y agroindustrias, la clasificación de frutas según calidad y tamaño es esencial para la cadena de suministro alimentaria. Actualmente este proceso se realiza de forma manual, lo que lo hace *lento, subjetivo y propenso a errores*, generando pérdidas económicas significativas, desperdicio de alimentos y dificultades para estandarizar la calidad del producto.

Este trabajo desarrolla un sistema capaz de analizar una fotografía de una fruta y asignar simultáneamente una categoría de calidad y una de tamaño en tiempo real. Las contribuciones principales son: (i) construcción de un dataset triclase que incluye una clase *Regular* construida manualmente; (ii) corrección del desbalanceo

entre clases ($IR \approx 3.14$) mediante pesos balanceados; (iii) identificación y corrección de un bug crítico de normalización en la CNN que causaba colapso de validación ($val_accuracy \approx 20\%$); y (iv) despliegue completo en Streamlit con selección de modelo.

II. Fundamentos Teóricos

A. Formulación matemática del problema

El problema se formula como una función de clasificación multi-salida $f: X \rightarrow Y_1 \times Y_2$, donde X es el espacio de entrada de la imagen, $Y_1 = \{0, 1, 2\}$ representa las tres clases de calidad (Mala, Buena, Regular) e $Y_2 = \{0, 1, 2\}$ las tres clases de tamaño (Pequeño, Mediano, Grande).

Para los modelos de ML, el vector de características $X \in \mathbb{R}^{513}$ se compone de un histograma de color HSV tridimensional ($8 \times 8 \times 8 = 512$ bins, normalizado $L2$) que captura la distribución cromática de la fruta independientemente de su orientación, más el área relativa de la fruta:

$$area_ratio = A_fruta / (H \times W)$$

donde H y W son las dimensiones de la imagen. Para la CNN, X es un tensor ($224 \times 224 \times 3$) normalizado con los parámetros z-score de ImageNet:

$$x' = (x - \mu) / \sigma, \text{ con } \mu = [0.485, 0.456, 0.406] \text{ y } \sigma = [0.229, 0.224, 0.225]$$

Esta normalización es idéntica en el preprocesamiento del dataset (preprocess.py), en las funciones de inferencia (helpers.py) y en el generador de datos de la CNN, lo que garantiza consistencia entre entrenamiento e inferencia.

B. Modelos de ML tradicional

Random Forest: ensamble de T árboles con predicción por votación mayoritaria. Desbalanceo corregido con $\text{class_weight}='balanced' \rightarrow w_j = n/(k \cdot n_j)$.

XGBoost: optimiza iterativamente una función de pérdida regularizada $L(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k)$, donde $\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$ controla la complejidad del árbol evitando sobreajuste. El desbalanceo se corrigió con sample_weight calculado con $\text{compute_class_weight}('balanced')$.

SVM lineal: maximiza el margen entre clases en el espacio HSV 512-dimensional. Desbalanceo corregido con $\text{class_weight}='balanced'$. Los tres modelos fueron entrenados con una partición train/test consistente sin búsqueda de hiperparámetros adicional en la ejecución final reportada

C. Red Neuronal Convolutacional (CNN)

La arquitectura diseñada desde cero consta de tres bloques convolucionales (Conv2D con 32, 64 y 128 filtros + BatchNormalization + MaxPooling2D), seguidos de GlobalAveragePooling2D y Dropout (0.5). Cada rama densa incluye BatchNormalization antes del Dropout (0.4) que producen dos salidas softmax simultáneamente:

- **quality_output:** 3 neuronas para las categorías Mala, Buena, Regular.
- **size_output:** 3 neuronas para las categorías Pequeño, Mediano, Grande.

La función de pérdida total es la suma de las pérdidas de entropía cruzada categórica escasa de cada rama: $L_{\text{total}} = L_{\text{calidad}} + L_{\text{tamaño}}$. El entrenamiento usó el optimizador Adam ($\text{lr} = 0.0001$) con tres callbacks: EarlyStopping ($\text{patience} = 5$, restaura mejores pesos), ReduceLROnPlateau ($\text{factor} = 0.5$, $\text{patience} = 3$) y ModelCheckpoint. Se aplicó class_weight por salida para compensar el desbalanceo.

D. Métricas de evaluación

Dado el desbalanceo entre clases ($IR \approx 3.14$), la accuracy global puede ser engañosa — un modelo que siempre prediga 'Good' obtendría 51.4% sin aprender nada. Por ello se priorizó el F1-macro, que promedia el F1-score de cada clase con igual peso, penalizando el bajo desempeño en clases minoritarias:

$$\text{F1_macro} = (1/K) \sum_k 2 \cdot P_k \cdot R_k / (P_k + R_k)$$

donde P_k y R_k son la precisión y el recall de la clase k . Se complementa con matrices de confusión para identificar patrones de error específicos por clase.

III. Metodología (CRISP-DM)

1. Comprensión del negocio

El objetivo de negocio es reducir las pérdidas económicas y el desperdicio alimentario en mercados y agroindustrias mediante la clasificación automática simultánea de calidad y tamaño de frutas. El criterio de éxito técnico definido fue: $\text{F1-macro} \geq 0.85$ en calidad para el mejor modelo, y un sistema de despliegue funcional accesible desde interfaz web.

2. Comprensión de los datos (EDA)

El dataset base proviene de Kaggle (Fruit Quality Classification [1]), que contiene imágenes de frutas de buena y mala calidad. El equipo construyó manualmente la clase Regular a partir de la carpeta 'mix quality_fruits' del mismo dataset y fotos propias recolectadas en hogares y mercados locales. El dataset final comprende 28 395 imágenes originales de seis tipos de fruta:

- **Clases de calidad:** Bad (9 142 imágenes, 32.2%), Good (14 599, 51.4%), Regular (4 654, 16.4%).
- **Frutas:** Apple (3 750), Banana (4 036), Guava (3 694), Lime (3 273), Orange (3 201), Pomegranate (10 441).
- **Dimensiones:** media 693×678 píxeles; alta variabilidad (rango 100 a 8 000 px), lo que justifica el redimensionado a 224×224 en el preprocesamiento.
- **Desbalanceo:** Imbalance Ratio (IR) = $14\,599 / 4\,654 \approx 3.14$, lo que requiere corrección explícita en todos los modelos.

Los histogramas RGB por clase mostraron que la clase Regular presenta distribuciones más planas y menos discriminativas que Bad y Good, anticipando su mayor dificultad de clasificación. La Figura 1 muestra la distribución completa del dataset.

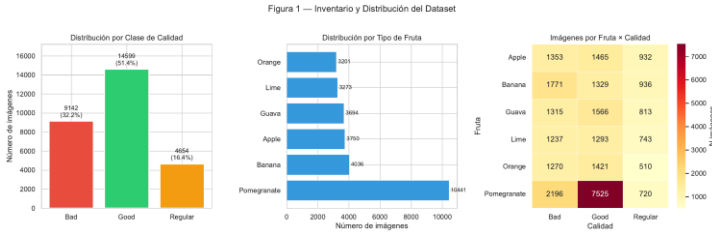


Figure 1: Distribución por clase de calidad, tipo de fruta e imagen por fruta $\times$ calidad.

3. Preparación de los datos

El pipeline de preprocesamiento (implementado en `src/data_processing/preprocess.py`) aplica las siguientes transformaciones sobre cada imagen original:

- **Remoción del fondo:** umbral HSV para segmentar la fruta del fondo uniforme.
- **Redimensionado:** todas las imágenes se reescalan a 224×224 píxeles (requerimiento de la CNN).
- **Normalización z-score de ImageNet:** $x' = (x/255 - \mu) / \sigma$, aplicada de forma idéntica en `preprocess.py`, `helpers.py` y el generador de la CNN.
- **Estimación de tamaño:** se calcula el `area_ratio` de cada imagen para asignar la etiqueta de tamaño (small/medium/large) en función de percentiles.
- **Augmentación $\times 4$:** flip horizontal/vertical, rotación y ajuste de brillo, aplicados SOLO sobre imágenes de entrenamiento.

La Figura 2 muestra el pipeline visual paso a paso. El dataset procesado final cuenta con 28 395 originales + 113 580 aumentadas (solo en train), como se aprecia en la Figura 3.

Estrategia anti-fuga de datos: El split para la CNN se realizó en tres conjuntos sobre imágenes originales: 70% train, 15% validación y 15% test. Las imágenes aumentadas se incorporan solo al conjunto de entrenamiento. Esto garantiza que el conjunto de prueba esté libre de contaminación y que las métricas reflejen la capacidad de generalización real de los modelos.



Figure 2: Pipeline de preprocesamiento: Original $\rightarrow$ Sin fondo $\rightarrow 224 \times 224 \rightarrow$ Normalizada.

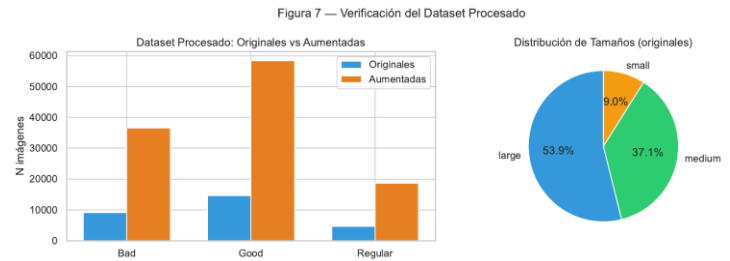


Figure 3: Dataset procesado: originales vs aumentadas por clase, y distribución de tamaños.

4. Modelado

ML tradicional (notebook 02): se entrenaron tres modelos base (Random Forest, SVM lineal, XGBoost) con MultiOutputClassifier bajo la misma partición train/test. Cada modelo fue configurado con pesos balanceados para compensar el desbalanceo de clases. Los modelos base se comparan directamente sin búsqueda adicional de hiperparámetros en la ejecución final reportada.

CNN (notebook 03): arquitectura bifurcada entrenada desde cero sin transfer learning. El generador `tf.data` aplica z-score de ImageNet de forma consistente con `preprocess.py`. Corrección crítica: un error de normalización previo (usar solo `/255` sin z-score) causaba `val_quality_accuracy` $\approx 20\%$. Una vez corregido, el modelo alcanzó 78%. Entrenó 41 épocas hasta Early Stopping.

5. Evaluación

Todos los modelos se evaluaron sobre el conjunto de prueba ($n = 5$ 679 imágenes para ML; $n = 4$ 260 para CNN), compuesto exclusivamente por imágenes originales no vistas durante el entrenamiento.

6. Despliegue

Interfaz Streamlit (`app.py`) que permite seleccionar entre cinco modelos, subir una imagen o capturar con cámara, y obtener predicción simultánea de calidad y tamaño con porcentaje de confianza. Los modelos se cargan desde `saved_models/` sin necesidad de re-entrenar.

[Video de demostración del proyecto \(10 min\)](#)

El video cubre: contexto del problema, recorrido por el repositorio, demostración del EDA, resultados de los modelos y demostración en vivo de la aplicación Streamlit.

IV. Resultados

A. Modelos de ML tradicional

La Tabla I resume el desempeño de los tres modelos sobre el conjunto de prueba (n = 5 679 imágenes originales). Los modelos se entrenaron con los hiperparámetros optimizados definidos directamente: Random Forest con `n_estimators = 200`, `class_weight = 'balanced'`; XGBoost con `n_estimators = 300`, `max_depth = 6`, `learning_rate = 0.05`. La búsqueda de hiperparámetros con RandomizedSearchCV está implementada en el notebook pero fue desactivada en el entrenamiento final (RUN_RF_TUNING = False) dado el alto costo computacional.

TABLA 1

Comparación de modelos de ML tradicional sobre el conjunto de prueba

Modelo	Acc. Calidad	F1 Calidad	Acc. Tamaño	F1 Tamaño	F1 Promedio
XGBoost	97.4 %	97.1 %	96.4 %	95.3 %	96.2 %
Random Forest	97.9 %	97.6 %	95.4 %	94.2 %	95.9 %
SVM Lineal	87.7 %	86.7 %	97.3 %	96.0 %	91.4 %

XGBoost obtuvo el mayor F1-macro promedio multi-salida (0.9616), combinando F1-macro de calidad = 0.9706 y F1-macro de tamaño = 0.9525. **Random Forest** superó a XGBoost en calidad individual (F1 = 0.9763 vs 0.9706), pero quedó por debajo en tamaño (0.9420 vs 0.9525), resultando en un F1 promedio levemente menor (0.9591). **SVM lineal** fue el modelo más rápido de comprender en términos de tamaño (F1 = 0.9605), pero el más débil en calidad (F1 = 0.8668), con un tiempo de entrenamiento de 528 s. XGBoost fue seleccionado como mejor modelo por presentar el mayor balance global entre las dos tareas.

La Figura 4 muestra las matrices de confusión del XGBoost. La clase *Regular* es la de mayor dificultad: menor soporte (931 muestras vs 1 828 de Mala y 2 920 de Buena) y naturaleza ambigua como zona de transición entre las otras dos clases.

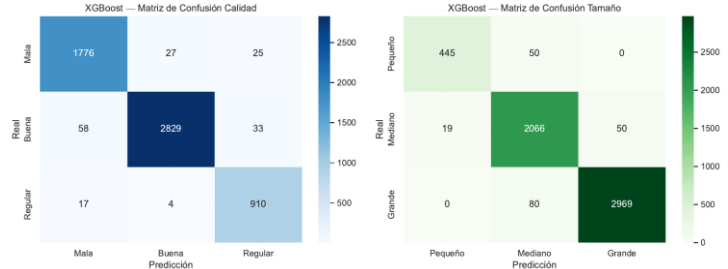


Figure 4: Matrices de confusión del modelo XGBoost (mejor modelo general).
Izquierda: predicción de calidad. Derecha: predicción de tamaño.

B. CNN multi-salida

La CNN fue entrenada durante 41 épocas (Early Stopping detuvo el proceso al no mejorar val_loss por 5 épocas consecutivas). La Tabla II detalla el reporte de clasificación sobre el conjunto de prueba (n = 4 260 imágenes).

TABLA 2

Reporte de clasificación de la CNN bifurcada (n = 4 260 imágenes)

Tarea	Clase	Precisión	Recall	F1-score
Calidad	Mala	88 %	64 %	74 %
Calidad	Buena	73 %	97 %	83 %
Calidad	Regular	93 %	47 %	62 %
Calidad	Macro avg	85 %	69 %	73 %
Tamaño	Pequeño	95 %	75 %	84 %
Tamaño	Mediano	86 %	96 %	91 %
Tamaño	Grande	98 %	93 %	96 %
Tamaño	Macro avg	93 %	88 %	90 %

V. Análisis de Resultados

A. Comparación ML tradicional vs CNN

Los modelos de ML tradicional superan a la CNN en calidad (97.9% vs 78% de accuracy). Esta diferencia responde a tres factores: (i) el histograma HSV 512-dimensional captura la

distribución cromática completa — manchas, oxidación, madurez — en un vector que los árboles de decisión y boosting aprovechan eficientemente; (ii) la CNN fue entrenada desde cero sin transfer learning, lo que limita su capacidad con 28 395 imágenes; (iii) el desbalanceo afecta más a la CNN, donde el `class_weight` por salida de Keras es menos directo que el `sample_weight` explícito de XGBoost.

La CNN, en cambio, aprende representaciones desde píxeles sin ingeniería manual de características, lo que le confiere mayor potencial de generalización a nuevas frutas y condiciones de iluminación.

B. Generalización y sobreajuste

Para auditar sobreajuste, se comparó el desempeño en prueba con las métricas de validación y se revisaron matrices de confusión por tarea. En ML tradicional, la cercanía entre XGBoost (0.9616) y Random Forest (0.9591) sugiere que el desempeño no depende de un único modelo y que ambos generalizan bien. En CNN, la convergencia estable del `val_loss` hasta la época 41 sin divergir del `train_loss` descarta sobreajuste severo. La diferencia entre tamaño (93%) y calidad (78%) indica que la red aprende mejor la señal geométrica (`area_ratio`) que los patrones sutiles de deterioro — especialmente en la clase Regular, cuyo recall es solo 47%.

C. Insights clave del proyecto

- **Regular es el cuello de botella de todas las familias de modelos.** Con 4 654 imágenes (16.4%) y naturaleza ambigua como zona de transición, es la clase con menor F1 en todos los modelos. Ampliarla mejoraría el IR.
- **La normalización z-score consistente fue crítica.** Antes de la corrección, `val_quality_accuracy` $\approx$ 20% (equivalente al azar para 3 clases). Una vez corregido el generador CNN para usar z-score de ImageNet, el modelo alcanzó 78%.
- **`area_ratio` con umbrales fijos es altamente efectivo para tamaño.** Todos los modelos superaron 93% en accuracy de tamaño. Sin embargo, la regla fija (< 0.20 / < 0.50 / ≥ 0.50) produce una distribución sesgada hacia 'large' (54%), lo que representa una limitación para escenarios donde el tamaño relativo de la fruta a la cámara varía significativamente.
- **XGBoost vs Random Forest: el ganador depende de la tarea.** RF supera a XGBoost en calidad individual, pero XGBoost tiene mejor balance global. Para una aplicación que prioriza calidad, RF sería preferible; para balance multi-tarea, XGBoost.

D. Comparación con la literatura

Trabajos como Kang et al. [6] reportan 94–98% de accuracy para clasificación de calidad de frutas con CNNs preentrenadas (ResNet, VGG16). La diferencia con nuestro resultado (78%) es consistente y esperada dado que no usamos transfer learning. Esta comparación reafirma que transfer learning es la mejora más prometedora para trabajo futuro.

VI. Aspectos Éticos – PI1 (Código de Ética Profesional IEEE)

A lo largo del desarrollo del proyecto **sosaSlaves** se identificaron diversas situaciones con implicaciones éticas. Una de las principales fue el posible **sesgo de clasificación**, ya que los modelos podrían clasificar con menor precisión frutas provenientes de regiones subrepresentadas en el conjunto de datos. Esta situación se relaciona con los principios de justicia e imparcialidad del código IEEE-CS, por lo que se adoptaron medidas como el uso de `class_weight='balanced'` y la recolección propia de imágenes más diversificadas.

También se consideró la **privacidad de los datos**, debido a que el dataset utilizado proviene de Kaggle y de fotografías propias tomadas por el equipo. Al no incluir datos personales identificables, se garantiza el respeto a la privacidad. Como medida ética, se utilizó exclusivamente información de dominio público y datos obtenidos con consentimiento.

Otra decisión importante fue asegurar la **transparencia del modelo**, ya que los sistemas de clasificación pueden dificultar la explicación de sus decisiones a los operarios de planta. Esta situación se vincula con los principios de integridad técnica y honestidad, por lo que se documentaron las métricas, las limitaciones del sistema y el proceso CRISP-DM completo.

Asimismo, se analizó el posible **impacto laboral** del proyecto, considerando que la automatización podría desplazar a clasificadores manuales. Este aspecto está relacionado con el bienestar social y la responsabilidad profesional. Por esta razón, el sistema se plantea como una herramienta de apoyo para los operarios y no como un reemplazo total del trabajo humano.

Finalmente, se tuvo en cuenta la **seguridad alimentaria**, ya que una clasificación incorrecta podría representar un riesgo para los consumidores. Este punto se relaciona con el principio de seguridad y bienestar público. Para mitigar este riesgo, se propuso validar el sistema mediante métricas rigurosas y permitir el uso de un umbral de confianza configurable dentro de la aplicación.

Todas las decisiones de diseño documentadas — split anti-fuga, pesos balanceados, uso de datos de dominio público, transparencia en métricas y limitaciones — reflejan el cumplimiento activo de los principios de *justicia, integridad técnica y responsabilidad pública* del código de ética IEEE-CS (Indicador de desempeño PI1).

VII. Análisis de Impactos — PI2 (Social, Ambiental, Económico, Global)

La matriz de impactos SAEG del sistema de clasificación permite analizar el proyecto desde cuatro dimensiones principales: social, económica, ambiental y global. En la dimensión **social**, el sistema genera un impacto positivo al reducir el desperdicio de alimentos y estandarizar la calidad de forma objetiva y trazable. Sin embargo, existe el riesgo de que la instalación de cámaras en el entorno de trabajo sea percibida como una forma de vigilancia. Para mitigar este riesgo, se debe comunicar explícitamente el propósito del sistema y garantizar que no se almacenen imágenes de los usuarios.

En la dimensión **económica**, el sistema puede contribuir a la reducción de costos operativos en cadenas de empaque, además de ser escalable a grandes volúmenes de producción sin requerir una inversión proporcional. No obstante, se identifica como riesgo la inversión inicial en hardware y una posible dependencia tecnológica. Como estrategia de mitigación, se propone un diseño de bajo costo basado en CPU y Streamlit, además del uso de código abierto.

En la dimensión **ambiental**, el principal impacto positivo consiste en que una menor cantidad de desperdicio alimentario contribuye a reducir la huella de carbono en la cadena de suministro. Aun así, se reconoce como riesgo el consumo energético asociado a la fase de entrenamiento del modelo, especialmente si se utilizan recursos como GPU. Para reducir este impacto, se plantea realizar un entrenamiento único, utilizar inferencia ligera en CPU y emplear modelos compactos.

Finalmente, en la dimensión **global**, la metodología CRISP-DM y el código documentado permiten que el sistema sea replicable en otros países y con distintos tipos de fruta. Sin embargo, pueden

presentarse diferencias culturales en los criterios de calidad, así como variaciones en las regulaciones de cada contexto. Para mitigar estos riesgos, se propone que las etiquetas de clase sean configurables y que la documentación esté disponible tanto en español como en inglés.

VIII. Conclusiones y Trabajo Futuro

Conclusiones

- **XGBoost es el mejor modelo global** (F1-macro promedio 0.9616), con el mejor balance entre calidad (0.9706) y tamaño (0.9525). Random Forest es preferible si la prioridad es calidad individual (F1 = 0.9763).
- **La CNN desde cero** ofrece resultados competitivos en tamaño (F1-macro 0.90) pero limitados en calidad (F1-macro 0.73). Transfer learning es la mejora más prometedora.
- **La consistencia del pipeline z-score** entre preprocess.py, helpers.py y el generador CNN fue la corrección técnica más crítica del proyecto.
- **CRISP-DM estructuró el proyecto** de forma iterativa, permitiendo detectar y corregir normalización, desbalanceo y fuga de datos antes de la evaluación final.
- **Criterio de éxito superado:** F1-macro calidad XGBoost = 0.9706 (objetivo ≥ 0.85) y sistema de despliegue funcional.

Trabajo Futuro

- Aplicar **transfer learning** (MobileNetV3, EfficientNet-B0) para mejorar la CNN en clasificación de calidad, esperando alcanzar F1-macro > 0.92 .
- Ampliar la clase **Regular** con al menos 3 000 imágenes adicionales recolectadas en plazas de mercado para reducir el IR.
- Revisar los umbrales fijos de area_ratio para tamaño o reemplazarlos por percentiles adaptativos según la fruta y condición de captura.
- Implementar Grad-CAM para visualizar qué regiones determinan la predicción de calidad (explicabilidad).

IX. Referencias

- [1] D. Park, 'Fruit Quality Classification Dataset,' Kaggle, 2023. Disponible: <https://www.kaggle.com/datasets/ryandpark/fruit-quality-classification>
- [2] T. Chen y C. Guestrin, 'XGBoost: A Scalable Tree Boosting System,' Proc. 22nd ACM SIGKDD, 2016, pp. 785–794.
- [3] L. Breiman, 'Random Forests,' Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
- [4] C. Cortes y V. Vapnik, 'Support-vector networks,' Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.
- [5] D. P. Kingma y J. Ba, 'Adam: A Method for Stochastic Optimization,' ICLR, 2015.
- [6] W. Kang et al., 'Fruit quality grading based on deep CNN,' IEEE Trans. Instrumentation and Measurement, vol. 70, 2020.
- [7] IEEE, 'IEEE Code of Ethics,' 2020. Disponible: <https://www.ieee.org/about/corporate/governance/p7-8.html>
- [8] P. Chapman et al., 'CRISP-DM 1.0: Step-by-step data mining guide,' SPSS Inc., 2000.
- [9] F. Pedregosa et al., 'Scikit-learn: Machine Learning in Python,' JMLR, vol. 12, 2011.
- [10] M. Abadi et al., 'TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems,' 2015.